

## Trabajo Práctico N° 3

### Expresiones e Instrucciones

**Lectura:** Sebesta Caps. 7 y 8, Scott Cap. 6

#### Conceptos

1. Defina las nociones de precedencia y asociatividad. ¿Existe algún otro mecanismo provisto por los lenguajes para especificar o modificar las reglas de precedencia y asociatividad?
2. Defina los conceptos de *efectos colaterales* y *transparencia referencial*, indique su relación e ilustre con ejemplos.
3. Defina y explique las diferencias entre los conceptos de evaluación *estricta*, *no estricta*, *perezosa* y con *cortocircuito*. ¿Puede un lenguaje de programación contar con más de un tipo de evaluación? Justifique su respuesta.
4. Analice la relación entre efectos colaterales y evaluación con cortocircuito. Ilustre, mediante un ejemplo, los problemas que pueden surgir.
5. Discuta acerca de la conmutatividad de los operadores aritméticos binarios como el  $+$  o el  $*$ . Analice la relación que pueden tener con los efectos colaterales.
6. Analice las ventajas y desventajas de:
  - a. Expresiones mixtas.
  - b. Operadores sobrecargados.
  - c. Transparencia referencial.
7. ¿Cómo impacta el orden de evaluación de expresiones en la semántica de un lenguaje? ¿Qué características y/o criterios de evaluación se ven afectados por este impacto? Complemente su respuesta analizando la función  $f$  que se muestra debajo (suponiendo que se usa en un programa como el de su derecha), determinando los valores para `sum_1` y `sum_2` en los siguientes casos: (i) los operadores son evaluados de izquierda a derecha; (ii) los operadores son evaluados de derecha a izquierda.

```
function f (var k:integer):integer;      ...
begin                                   i := 10;
    k := k + 4;                          sum_1 := (i/2) + f(i);
    f := 3 * k;                          j := 10;
end;                                     sum_2 := f(j) + (j/2);
```

8. Algunos lenguajes permiten que la asignación pueda ser utilizada como un operador más en las expresiones. ¿Cuáles son las ventajas y las desventajas de esta decisión? Mencione un lenguaje donde esto sea posible e ilustre su uso con un ejemplo.
9. Defina sobrecarga dependiente e independiente del contexto. Analice las posibles ventajas y desventajas de un lenguaje que brinde la posibilidad de usar ambas formas de sobrecarga.

10. Considerando las porciones de código que se muestran a la derecha, en cada caso muestre el valor que queda asociado a `w`, usando las formas de evaluación que se indican a continuación. En cada caso, justifique su respuesta.
- Evaluación ansiosa (estricta).
  - Evaluación no estricta (simple).
  - Evaluación perezosa.
- |                                                                                                                                                                        |                                                                                                                                                                                                     |                                                                                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>int x,w,z; x = 1; z = 1; w = f(x+z);  int f (int y){     z = 10     if(y &gt; 5) {         x = 10;         return y;     } else {         return 0;     } }</pre> | <pre>int x,w,z, p; x = 1; z = 1; p = 1; w = f(x+z+p);  int f (int y){     int p = 10;     z = 10;     if(y &gt; 5) {         x = 10;         return y;     } else {         return 0;     } }</pre> | <pre>function f(int x, y, z): int {     if (z &gt; 0)         then return x + y         else return 2*y - a }  ... int a = 20; ... int w = f(g(), h(), i()); ...</pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
11. En el siguiente código, explique qué tipo de evaluación (ansiosa, no estricta, perezosa, con cortocircuito) conviene adoptar. Si considera que más de un tipo de evaluación resulta adecuado, indique las diferencias que se producen. (Considere que las funciones `g()`, `h()` e `i()` retornan enteros.)
12. Indique cómo pueden simularse cada una de las estructuras de control a nivel instrucción propuestas por el principio de programación estructurada usando únicamente secuencia y transferencia de control. Analice los riesgos y ventajas que provoca el uso del *goto*. ¿Cómo se incorpora esta instrucción en los lenguajes actuales?
13. La semántica del constructor `for` puede variar de un lenguaje a otro (o hasta de una implementación a otra) según las restricciones que se impongan sobre la variable de control y las variables involucradas en las expresiones. Comente dos interpretaciones diferentes provistas por lenguajes de programación contemporáneos, y analice sus ventajas y desventajas. Luego muestre la salida del segmento de código que se muestra a la derecha, considerando las interpretaciones descriptas previamente.
- |  |                                                                                                                                                         |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <pre>j := 1; k := 20; for i := j+1 to k-1 do begin     write (1, i, j, k);     i := i+1;     j := j+1;     k := k-1;     write (2, i, j, k); end;</pre> |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------|
14. Considere un lenguaje que no tiene variables (solo maneja constantes). ¿Puede tal lenguaje tener algún mecanismo de iteración? Justifique, y en caso afirmativo ejemplifique.

## Casos de Estudio e Investigación

- Describa la semántica de los operadores `+=` y `++` de Java, y analice sus ventajas y desventajas.
- Explique en qué consiste una expresión condicional, e indique sus ventajas. Muestre un ejemplo a partir de un lenguaje que brinde esta característica.
- Compare los operadores lógicos `&`, `&&` de Java y `and` de Pascal, contrástelos mediante ejemplos.
- Hay lenguajes de programación que permiten utilizar estructuras de datos infinitas. ¿Qué tipo de evaluación es necesario para crearlas y manipularlas adecuadamente? ¿Por qué?
- ¿Debería permitirse a un optimizador de código para Java el cambio de orden entre sub-expresiones de una expresión booleana? Justifique y ejemplifique su respuesta.
- ¿Cómo se podrían evitar los efectos colaterales funcionales en un lenguaje como Pascal? ¿Qué desventajas traería realizar esto?

21. Muestre, siguiendo la sintaxis de Pascal, dos funciones donde se dé sobrecarga independiente del contexto, y dos funciones donde se dé sobrecarga dependiente del contexto. ¿Java permite ambos tipos de sobrecarga? ¿Por qué?
22. En Python ¿hay operadores sobrecargados? ¿Es posible definir funciones sobrecargadas? ¿Por qué?
23. Investigue en JavaScript las características del operador + y explique a qué resultado llegan las siguientes expresiones:

| Expresión    | Resultado | Expresión              | Resultado |
|--------------|-----------|------------------------|-----------|
| +3           |           | 1 + 2                  |           |
| + "-3"       |           | 3.14 + 2               |           |
| + "3.14"     |           | "hola" + "mundo"       |           |
| + "3"        |           | 5 + "hola"             |           |
| +true        |           | Infinity + 5           |           |
| +false       |           | [1,2,3] + [4,5,6]      |           |
| +null        |           | true + true            |           |
| + "Infinity" |           | "5" + true             |           |
| + "infinity" |           | + "5" + true           |           |
| +NaN         |           | "b" + "a" + +"a" + "a" |           |

Según lo investigado/observado, este operador ¿permite expresiones mixtas? ¿Está sobrecargado? ¿Tiene coerción? ¿Es ortogonal en el comportamiento binario respecto al unario? Justifique.

24. Investigue qué diferencia hay entre el operador == y el === en JavaScript. ¿Cuál se parece más al operador == de Java? ¿Es exactamente el mismo?
25. Determine a qué resultado llegan las siguientes secuencias de expresiones en JavaScript e investigue por qué llegan a ellos:
  - a) == 0  
0 == "0"  
== "0"
  - b) undefined == false  
false == null  
undefined == null
  - c) NaN === NaN  
NaN == NaN
  - d) !!false == !!true
26. Compare el constructor for de Pascal, Java y Python. Mencione al menos un aspecto positivo y uno negativo de cada uno. Ilustre la comparación con ejemplos.
27. Tanto Go como C incluyen el constructor goto. Investigue las limitaciones que cada lenguaje impone sobre su uso y responda: ¿por qué cree que los lenguajes modernos siguen incluyendo este constructor, a pesar de las críticas históricas que recibió?
28. Muestre la salida de las siguientes porciones de código considerando la semántica de los lenguajes Java, e indique cuál es el valor final de la variable i.

```
main() {
    int i;
    for (i=0; i<10; i++) {
        if (i == 5)
            break;
        System.out.print(i);
    }
    System.out.print("Fuera del bucle con valor i:" + i);
}
```

```
main () {
    int i;
    for (i=0; i<10; i++) {
        if (i == 5)
            continue;
        System.out.print(i);
    }
    System.out.print("Fuera del bucle con valor i:" + i);
}
```

29. Investigue si algún lenguaje de programación brinda la posibilidad de utilizar una etiqueta acompañando a la instrucción break. Muestre la utilidad de esta característica con un ejemplo.

30. En Python tanto la sentencia `while` como la sentencia `for` pueden contar con una cláusula `else`. Investigue el funcionamiento de tal cláusula para ambas sentencias y discuta su utilidad.
31. Describa y compare las facilidades para soportar iteradores (iteración en base a datos estructurados) de Java y Python. Puntualmente investigue acerca del constructor *yield* y la función predefinida *next* de Python, todo esto usado en conjunto con el constructor **for .. in**. Puede utilizar este código en Python (código) para experimentar.
32. Analice las diferencias entre los constructores `for ... in` y `for ... of` de JavaScript. ¿Cuáles son las ventajas y/o desventajas de contar con estos dos constructores?
33. Dado un arreglo de 5 enteros en Go, (`s := []int{1, 2, 3, 4, 5}`), investigue el comportamiento del constructor *for* e implemente 3 maneras distintas de recorrer el arreglo e imprimir sus componentes uno a uno. Distinga las decisiones de diseño más importantes del constructor, y contrástelo frente al *for* de Python.
34. Compare la instrucción *switch* de Java con la instrucción *case* de Pascal, apelando a dos criterios de evaluación para realizar su análisis.
35. En Go, las instrucciones *if* y *switch* permiten incluir una sentencia de inicialización para variables antes de la expresión de control. Muestre ejemplos y luego compare estos constructores con sus análogos en Java, prestando especial atención al alcance de las variables involucradas y a las ventajas y limitaciones que pueden brindar las versiones de Go.

El `switch` de Go y el de Java adoptan convenciones opuestas respecto al fall-through. Analice los siguientes dos programas y explique por qué, a pesar de su similitud, producen resultados diferentes.

| Java                                                                                                                                                                               | Go                                                                                                                                                   |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>int x = 1; switch (x) {     case 1:         System.out.println("uno");     case 2:         System.out.println("dos");     case 3:         System.out.println("tres"); }</pre> | <pre>x := 1 switch x {     case 1:         fmt.Println("uno")     case 2:         fmt.Println("dos")     case 3:         fmt.Println("tres") }</pre> |